Lovely M. Eusebio

B04 – Data Analyst Professional

I. Project Overview: Retail Sales Performance & Customer Insights

Business context

The data comes from a Philippine multi-channel e-commerce retailer. Orders arrive through three channels (Web, Mobile App, Marketplace) and are paid by methods such as GCash and COD. Shipping runs through couriers such as J&T, NinjaVan and 2GO. Customers are spread across regions, from Metro Manila to Caraga, and are grouped into loyalty tiers (e.g. Bronze).

The dataset has seven linked tables:

Table	Rows (approx.)	Role
customers	5,000	Demographics, region/city, signup date, loyalty tier
products	800	Category, brand, unit cost and price, stock
orders	15,000	Order header: status, channel, shipping method, discount, shipping fee
order_items	37,700	Line items: quantity and price at purchase
payments	15,000	Payment method, status and amount paid
shipments	15,000	Courier, ship and delivery dates, shipment status
returns	1,200	Return reason, refund flag and amount

About 66% of orders are Completed, 12% Shipped, 10% Pending, 8% Cancelled and 4% Returned. Returns are spread evenly across seven reasons (changed mind, defective, missing parts, size issue, late delivery, wrong item, damaged item).

Objective

The project answers one question: **"How is the store performing, and what drives revenue?"** It has two deliverables:

1. **An analyst notebook (report).** It loads the CSVs, cleans them, builds a fact table, computes KPIs, produces 5+ charts, and finishes with 5–8 insights and 2 recommendations.
2. **An interactive Streamlit dashboard (app.py).** It lets stakeholders explore the same story themselves.

The dashboard's KPIs are:

- Gross Sales
- Fulfilled Orders
- Average Order Value (AOV), calculated at order level rather than line-item level
- Top Category

Its views are:

- Monthly sales trend
- Category comparison
- Region comparison
- Order value distribution
- Channel comparison
- Top products
- A filtered transaction table

The sidebar filters are month, region, channel and category.

Scope

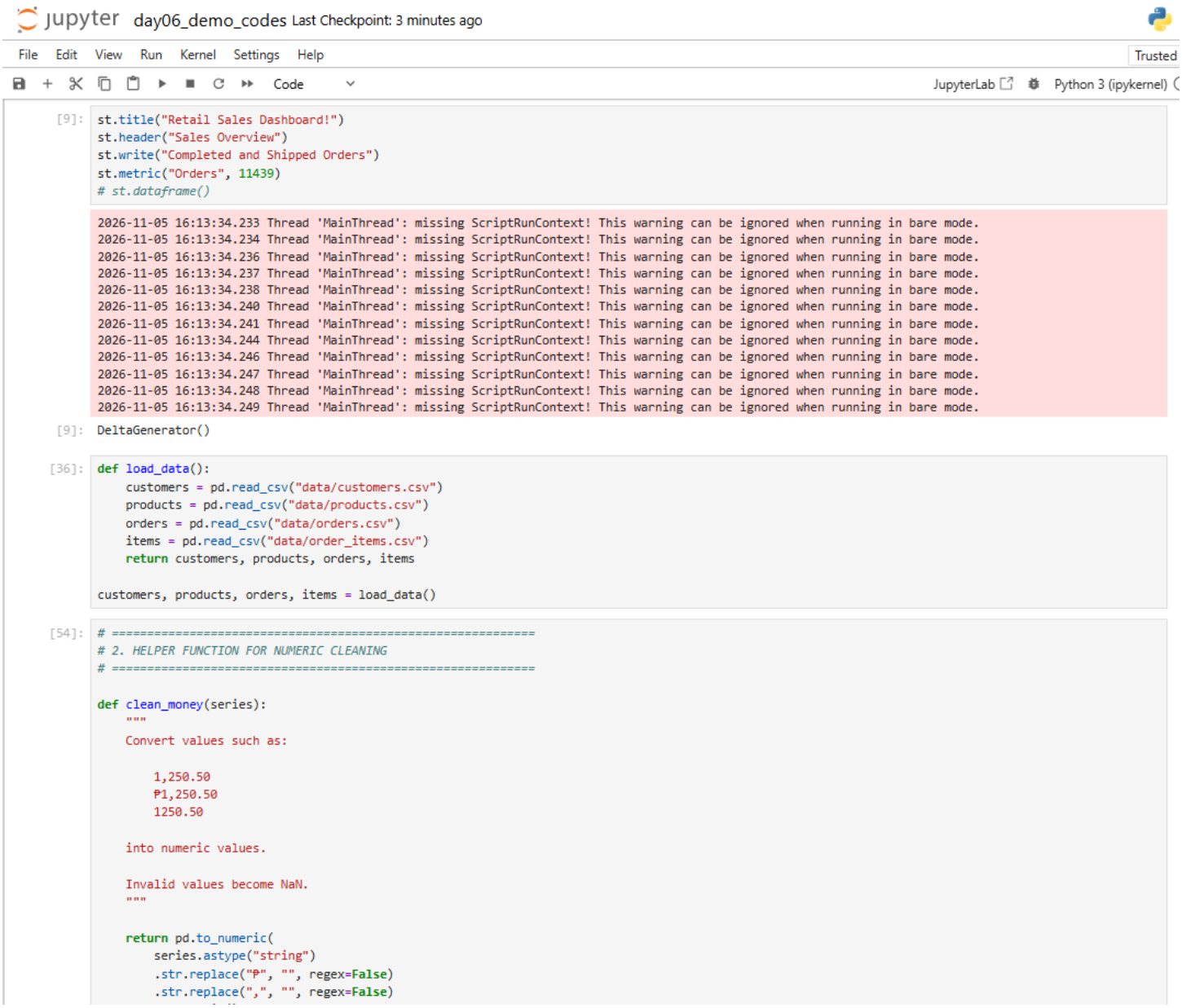
In scope

- **Cleaning:** The data is deliberately messy and needs standardizing.
- **Modelling:** Joining customers, products, orders and items into one analysis-ready fact table, with revenue defined consistently.
- **Analysis:** The dashboard analyses fulfilled orders only (Completed and Shipped), so Cancelled, Pending and Returned orders don't inflate sales.
- **Output:** KPIs, visuals, written insights and recommendations.

Currently out of scope or not yet used

- payments, shipments and returns are not loaded by the dashboard. That leaves out payment-method mix, delivery performance, and return and refund impact. These are natural extensions, and they add a profitability angle when combined with unit_cost.
- Forecasting, customer segmentation models and anything beyond descriptive and diagnostic analysis.

Jupyter Notebook Screenshots



jupyter day06_demo_codes Last Checkpoint: 3 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
# B. REMOVE EXACT DUPLICATES
# ------------------------------------------------------

customers = (
    customers
    .drop_duplicates()
    .copy()
)

products = (
    products
    .drop_duplicates()
    .copy()
)

orders = (
    orders
    .drop_duplicates()
    .copy()
)

items = (
    items
    .drop_duplicates()
    .copy()
)
```

[43]:
```
category_map = {
    "electronics": "Electronics",
    "elec": "Electronics",
    "home & kitchen": "Home & Kitchen",
    "home and kitchen": "Home & Kitchen",
    "books": "Books",
    "book": "Books",
    "groceries": "Groceries",
    "grocery": "Groceries",
    "toys": "Toys",
    "toy": "Toys",
    "beauty": "Beauty",
    "beauty & care": "Beauty",
    "fashion": "Fashion",
    "fashon": "Fashion",
    "sports": "Sports",
    "sports & outdoor": "Sports"
}

products["category_clean"] = (
    products["category"]
    .astype("string")
    .str.strip()
```

jupyter day06_demo_codes Last Checkpoint: 4 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
    "sports": "Sports",
    "sports & outdoor": "Sports"
}

products["category_clean"] = (
    products["category"]
    .astype("string")
    .str.strip()
    .str.lower()
    .replace(category_map)
    .fillna("Unknown")
)
```

[15]:
```
products
```

[15]:

	product_id	product_name	brand	category	unit_cost	unit_price	stock_qty	uom	launch_date	category_clean
0	PROD00001	KidJoy Beauty 963	KidJoy	Beauty	44.17	64.5	125.0	pc	2022-01-18	beauty
1	PROD00002	HomeEase Home 368	HomeEase	Home and Kitchen	37.18	70.39	131.0	pcs	2022-11-25	Home & Kitchen
2	PROD00003	FreshCart Fashion 911	FreshCart	Fashion	46.29	₱70.26	41.0	piece	2023-09-04	fashion
3	PROD00004	NovaTech Toys 753	NovaTech	Toys	43.66	62.46	149.0	pcs	2023-08-12	Toys
4	PROD00005	GlowCo Books 916	GlowCo	Books	64.59	82.02	162.0	bottle	2024-05-12	Books
...	...	...	...	...	...	...	...	...	...	...
795	PROD00796	KidJoy Elec 294	KidJoy	Elec	83.80	132.12	111.0	bottle	2023-08-15	Electronics
796	PROD00797	SunPeak Books 182	SunPeak	Books	53.67	62.54	163.0	bottle	2023-01-05	Books
797	PROD00798	NovaTech Groceries 889	NovaTech	Groceries	63.54	112.46	155.0	set	2022-08-09	Groceries
798	PROD00799	Acme Fashion 991	Acme	Fashion	25.27	34.96	169.0	bottle	2024-06-19	fashion
799	PROD00800	NovaTech Sports 419	NovaTech	Sports	70.14	116.39	245.0	piece	07/03/2022	sports

800 rows × 10 columns

[55]:
```
# ------------------------------------------------------
# D. CLEAN ORDER ITEM VALUES
# ------------------------------------------------------

items["quantity"] = pd.to_numeric(
    items["quantity"],
    errors="coerce"
)

items["unit_price_clean"] = clean_money(
    items["unit_price_at_purchase"]
)
```

jupyter day06_demo_codes Last Checkpoint: 4 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
# Keep valid quantity and price rows only
items = items[
    (items["quantity"] > 0)
    &
    items["unit_price_clean"].notna()
].copy()
```

[59]:

```
# ------------------------------------------------------------
# E. CLEAN ORDERS
# ------------------------------------------------------------

# Your dataset contains several date formats.
# format="mixed" allows Pandas to interpret them.

orders["order_date"] = pd.to_datetime(
    orders["order_datetime"],
    format="mixed",
    errors="coerce",
)

# Convert discount to numeric

orders["discount_pct"] = pd.to_numeric(
    orders["discount_pct"],
    errors="coerce",
)

# Missing discount means no discount

orders["discount_pct"] = (
    orders["discount_pct"]
    .fillna(0)
)

# Convert shipping fee

orders["shipping_fee"] = pd.to_numeric(
    orders["shipping_fee"],
    errors="coerce",
)

# Keep valid discount percentages only.
# Example:
# 0.10 = 10%
# 0.25 = 25%

orders = orders[
    orders["discount_pct"].between(0, 1)
].copy()
```

jupyter day06_demo_codes Last Checkpoint: 5 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[60]:

```
# ------------------------------------------------------------
# F. REFERENTIAL INTEGRITY
# ------------------------------------------------------------

# Remove orders whose customer_id does not exist
# in the customer master file.

orders = orders[
    orders["customer_id"].isin(
        customers["customer_id"]
    )
].copy()

# Remove order items whose product does not exist

items = items[
    items["product_id"].isin(
        products["product_id"]
    )
].copy()

# Remove order items whose order does not exist

items = items[
    items["order_id"].isin(
        orders["order_id"]
    )
].copy()
```

[61]:

```
# ------------------------------------------------------------
# G. BUILD ITEM-LEVEL ANALYTICAL DATAFRAME
# ------------------------------------------------------------

sales = (
    items
    .merge(
        orders,
        on="order_id",
        how="inner",
        validate="many_to_one",
    )
    .merge(
        products[
            [
                "product_id",
                "product_name",
                "brand",
                "category_clean",
            ]
        ],
```

jupyter day06_demo_codes Last Checkpoint: 5 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
        ],
        on="product_id",
        how="inner",
        validate="many_to_one",
    )
    .merge(
        customers[
            [
                "customer_id",
                "region_name",
                "city",
                "loyalty_tier",
            ]
        ],
        on="customer_id",
        how="left",
        validate="many_to_one",
    )
)
```

[62]:

```
# ------------------------------------------------------------
# H. CREATE ANALYTICAL FIELDS
# ------------------------------------------------------------

# Gross Sales
#
# Quantity × Unit Price

sales["gross_sales"] = (
    sales["quantity"]
    *
    sales["unit_price_clean"]
)

# Net Sales
#
# Gross Sales after discount

sales["net_sales"] = (
    sales["gross_sales"]
    *
    (
        1
        -
        sales["discount_pct"]
    )
)

# Create month column
```

jupyter day06_demo_codes Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
sales["net_sales"] = (
    sales["gross_sales"]
    *
    (
        1
        -
        sales["discount_pct"]
    )
)

# Create month column

sales["month"] = (
    sales["order_date"]
    .dt.to_period("M")
    .astype("string")
)
```

[63]:

```
# ------------------------------------------------------------
# I. KEEP ONLY FULFILLED ORDERS
# ------------------------------------------------------------

fulfilled_sales = sales[
    sales["order_status"].isin(
        [
            "Completed",
            "Shipped",
        ]
    )
].copy()

# Remove rows with unparseable dates

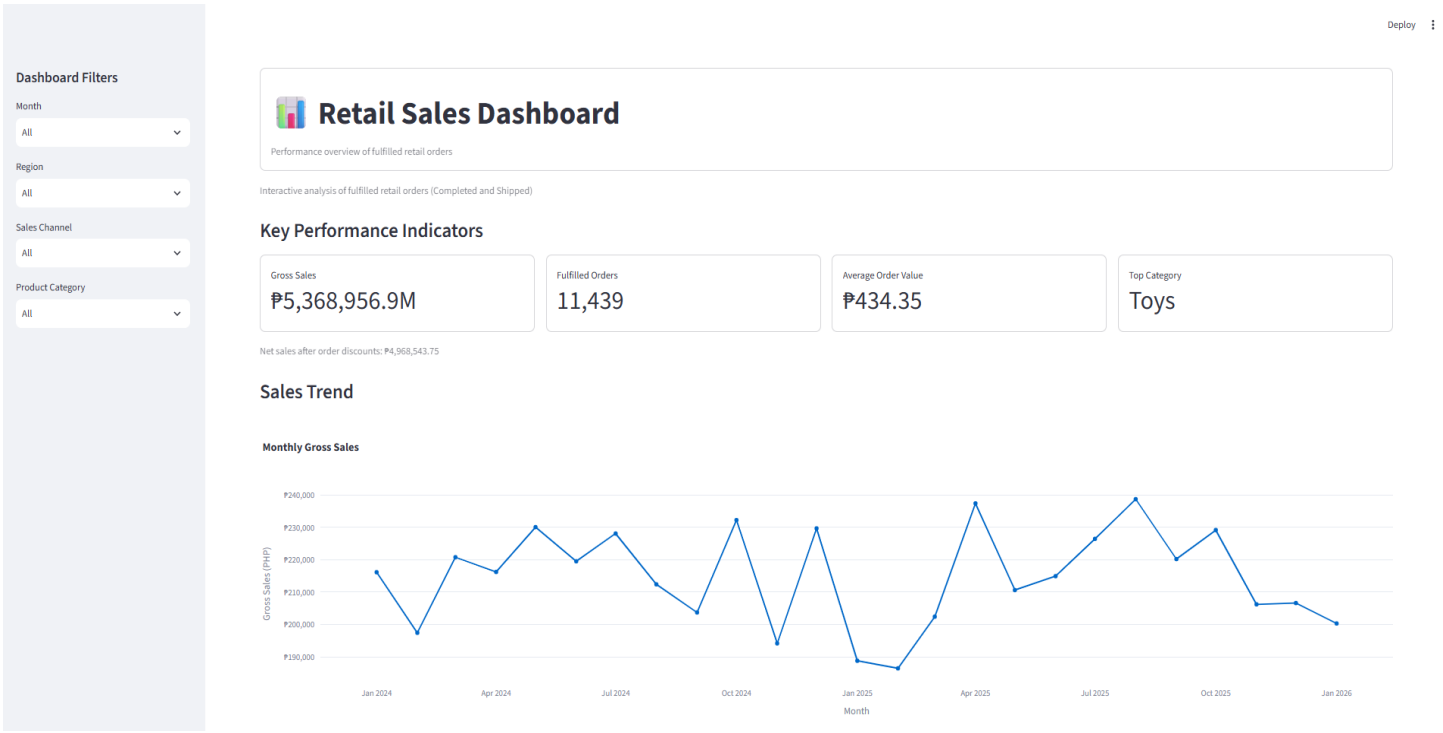
fulfilled_sales = fulfilled_sales[
    fulfilled_sales["order_date"].notna()
].copy()

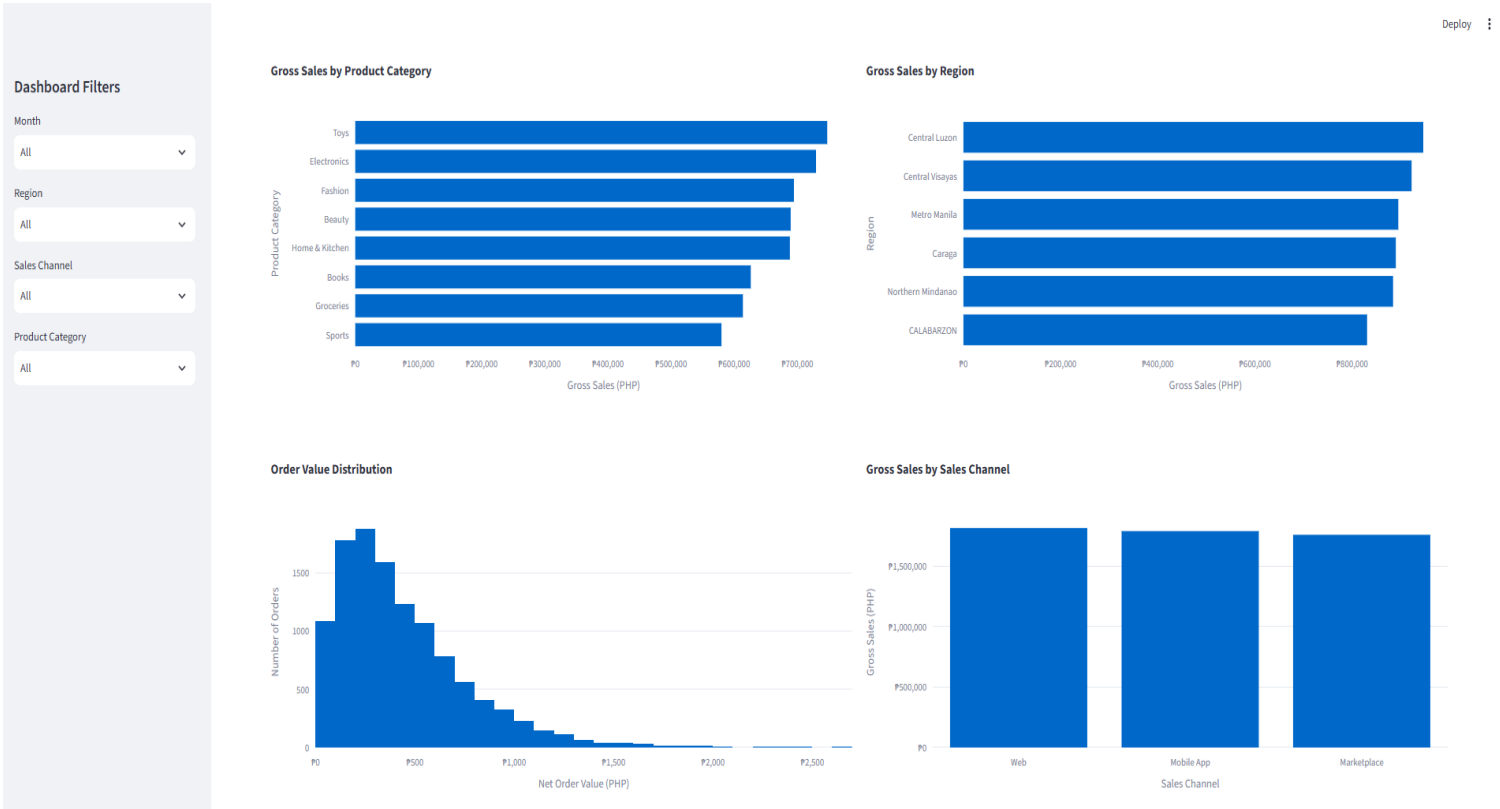
return fulfilled_sales
```

```
  Cell In[63], line 20
    return fulfilled_sales
    ^
SyntaxError: 'return' outside function
```

[ ]:

II. Dashboard Screenshot





Deploy

Dashboard Filters

Month: All

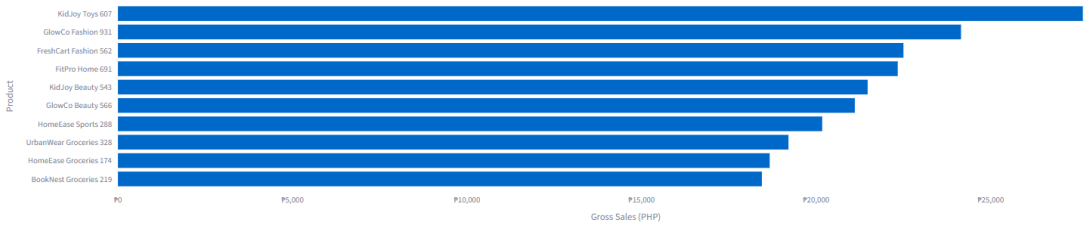
Region: All

Sales Channel: All

Product Category: All

Top Products

Top 10 Products by Gross Sales



View Filtered Transaction Details

order_id	order_date	product_name	category_clean	quantity	unit_price_clean	region_name	city	sales_channel	discount_pct	gross_sales	net_sales
ORD0001460	2026-01-31 23:36:00	FitPro Groceries 390	Groceries	1	117.74	Central Visayas	Cebu City	Web	0	117.74	117.74
ORD0001460	2026-01-31 23:36:00	Acme Home 695	Home & Kitchen	1	76.62	Central Visayas	Cebu City	Web	0	76.62	76.62
ORD0001460	2026-01-31 23:36:00	FreshCart Beauty 175	Beauty	1	97.69	Central Visayas	Cebu City	Web	0	97.69	97.69
ORD0001460	2026-01-31 23:36:00	BookNest Toys 579	Toys	1	134.22	Central Visayas	Cebu City	Web	0	134.22	134.22
ORD0005059	2026-01-31 22:08:00	SunPeak Groceries 527	Groceries	1	61.87	Caraga	Surigao City	Mobile App	0.075	61.87	57.2298
ORD0005059	2026-01-31 22:08:00	Acme Sports 177	Sports	2	23.64	Caraga	Surigao City	Mobile App	0.075	47.28	43.734
ORD0005059	2026-01-31 22:08:00	FreshCart Home 376	Home & Kitchen	2	192.58	Caraga	Surigao City	Mobile App	0.075	385.16	356.273
ORD0005059	2026-01-31 22:08:00	HomeEase Books 258	Books	1	85.54	Caraga	Surigao City	Mobile App	0.075	85.54	79.1245
ORD0014802	2026-01-31 21:58:00	FreshCart electronics 129	Electronics	1	154.7	CALABARZON	Cavite City	Marketplace	0.155	154.7	130.7215
ORD0014802	2026-01-31 21:58:00	Acme Toys 958	Toys	2	127.73	CALABARZON	Cavite City	Marketplace	0.155	255.46	215.8637

Deploy

Dashboard Filters

Month: All

Region: All

Sales Channel: All

Product Category: All

ORD0005059	2026-01-31 22:08:00	FreshCart Home 376	Home & Kitchen	2	192.58	Caraga	Surigao City	Mobile App	0.075	385.16	356.273
ORD0005059	2026-01-31 22:08:00	HomeEase Books 258	Books	1	85.54	Caraga	Surigao City	Mobile App	0.075	85.54	79.1245
ORD0014802	2026-01-31 21:58:00	FreshCart electronics 129	Electronics	1	154.7	CALABARZON	Cavite City	Marketplace	0.155	154.7	130.7215
ORD0014802	2026-01-31 21:58:00	Acme Toys 958	Toys	2	127.73	CALABARZON	Cavite City	Marketplace	0.155	255.46	215.8637

About this Dashboard

This dashboard uses the original retail CSV files.

The application performs basic cleaning before analysis:

- removes exact duplicate records
- standardizes product categories
- converts unit prices to numeric values
- removes zero or negative quantities
- parses mixed order-date formats
- fills missing discounts with zero
- validates customer, order, and product references
- keeps Completed and Shipped orders for sales analysis

Gross Sales:

Quantity × Unit Price

Net Sales:

Gross Sales × (1 − Discount Percentage)

Average Order Value is calculated at the order level, not at the order-item level.

Filtered item rows: 27,077

Filtered unique orders: 11,439

III. Data Analysis & Methodology

The analysis is descriptive. The raw CSVs are cleaned and joined into one line-item table, restricted to fulfilled orders, then aggregated into KPIs and charts.

Cleaning:

- Exact duplicates are removed.
- Category labels are standardized (e.g. "Fashon" becomes "Fashion").
- Prices like ₱70.26 are converted to numbers.
- Mixed date formats are parsed.
- Missing or invalid discounts are set to 0%.
- Orphan records are dropped.

Filters: Only Completed and Shipped orders are included, and users can narrow by month, region, channel and category.

Metrics:

- **Gross sales** = quantity × unit price at purchase.
- **Net sales** = gross sales × (1 − discount %).
- **Fulfilled orders** = distinct order IDs.
- **AOV** = net sales summed per order, then averaged across orders.
- **Top category** = the category with the highest gross sales.

Techniques: Grouped aggregations give the monthly trend and the category, region and channel comparisons. The rest are an order-value histogram, a top-10 product ranking and a drill-down transaction table.

IV. Key Takeaways

Total net revenue reached ₱5,919,125 across 15,075 orders with an overall profit margin of 17.86%, though order completion stands at 65.7% due to operational leakages.

- **Revenue & Volume Baseline:** The store processed 15,075 total orders generating ₱6,401,225 in gross sales and ₱5,919,125 in net sales (after discounts and adjustments), yielding ₱1,057,183 in total gross profit.

- **Order Fulfillment Bottleneck:** Only 65.7% of orders (9,903) reach Completed status. Cancelled orders represent 8.2% (1,238 orders / $472,586 net revenue), while pending and shipped orders account for the remaining active balance.

- **Balanced Channel Distribution:** Sales are evenly split across channels with Web leading net revenue at ₱1,990,030 (4,998 orders), Marketplace generating ₱1,981,268 (5,070 orders), and Mobile App contributing ₱1,947,826 (5,007 orders).

- **Margin Discrepancies & Data Categorization Spikes:** Categories like Grocery and Electronics generate strong margins (up to 31.95%), whereas Beauty & Care (5.81%) and Fashion (9.54%) suffer from cost-revenue misalignments or spelling variations (Fashon, electronics) in legacy records.

- **Logistics & Delivery Losses:** Out of 15,000 tracked shipments, 161 packages were lost (1.07%) and 465 were returned to sender (3.10%). Across courier services, J&T had the highest lost package count (38 lost, 109 returned to sender).

- **Return & Refund Erosion**: A total of 1,200 return requests were logged, resulting in ₱674,251 in refunds issued. The primary stated return drivers were Changed mind (182 requests), Defective items (178 requests), and Missing parts (175 requests**)**.

V. Insights

- **Fulfillment Pipeline Revenue Leakage:** With ₱472,586 in net sales tied up in cancelled orders and ₱241,065 in returned orders, unfulfilled or cancelled demand accounts for over 12% of gross revenue potential. Addressing order cancellation triggers can immediately capture high-margin revenue.

- **Courier Service Level Agreement (SLA) Parity:** Courier performance is tightly grouped, but logistics failures remain a primary friction point. J&T exhibits higher loss/RTS rates compared to competitors like 2GO (31 lost) or NinjaVan (25 lost).

- **Quality Control & Catalog Consistency:** Product defect issues (defective products and missing parts combined) drive 29.4% of total customer returns (353 of 1,200 returns), indicating underlying supplier quality issues rather than buyer remorse alone.

- **Multi-Channel Equalization:** Marketing spend and sales conversion are stable across Web, Mobile App, and Marketplace, showing that customers are multi-platform users with similar basket sizes (₱117 - ₱123 net average order value per channel).

VI. Summary

The business generated ₱5,919,125 in net revenue and ₱1,057,183 in gross profit across 15,075 total transaction records. While overall top-line volume across Web, App, and Marketplace channels remains balanced and consistent, profitability is constrained by an order completion rate of 65.7%. Operational frictions—including ₱674,251 in return refunds, 1,238 order cancellations, and 626 failed delivery attempt/lost shipments—represent the primary areas requiring intervention to expand gross margins beyond the current 17.86% benchmark.

VII. Conclusions

1. Order Completion is the Primary Margin Expansion Lever: The company loses significant upside to order drop-off prior to delivery completion (₱472k lost to cancellations).

2. Logistics Vulnerabilities Drive Capital Loss: Lost packages and returned-to-sender shipments directly increase inventory holding costs and write-offs across all 5 courier partners.

3. Product Sourcing & Quality Control Gaps: Returns driven by defect claims or missing parts disproportionately hit profitability through refund pay-outs (₱201.7k combined across defective and missing parts reasons).

VIII. Recommendations

1. **Standardize Catalog Taxonomy & Margin Floors:** Clean up duplicate categories in the database (e.g., merge Fashon into Fashion) and adjust pricing rules for low-margin lines like Beauty & Care to target a minimum 20% gross margin floor.

2. **Implement Courier Performance Audits:** Hold logistics providers (specifically J&T) accountable for the 161 lost packages and establish strict SLA penalty fees for lost inventory.

3. **Vendor Quality Audits for Return Mitigation:** Audit product lines associated with the 353 defective/missing part returns to enforce supplier warranties or re-negotiate vendor terms.